

TD1

Remarque :

La méthode **toString()** est une méthode de la classe Object en Java qui permet de retourner une chaîne de caractères représentant l'objet en question. Cette méthode est souvent redéfinie dans les classes pour retourner une chaîne de caractères personnalisés représentant l'état de l'objet.

Exercice1 :

1. Ecrire une classe Animal où un animal est caractérisé par son nombre de pattes, couleur et son poids. N'oubliez pas d'implémenter les constructeurs (avec argument(s) et sans arguments), les méthodes accesseurs, la méthode toString ainsi que la méthode boire.
2. A partir de cette classe de base, dérivez ensuite les classes Omnivore et Herbivore. Ainsi que la méthode jeMangeQuoi renvoyant une chaîne de caractères répondant à la question posée

```
void boire()
{
    //Affiche le message « Je bois de l'eau »
}

String toString()
{
    //retourne une chaine de caractères qui décrit cet animal (ses caractéristiques)}
}
```

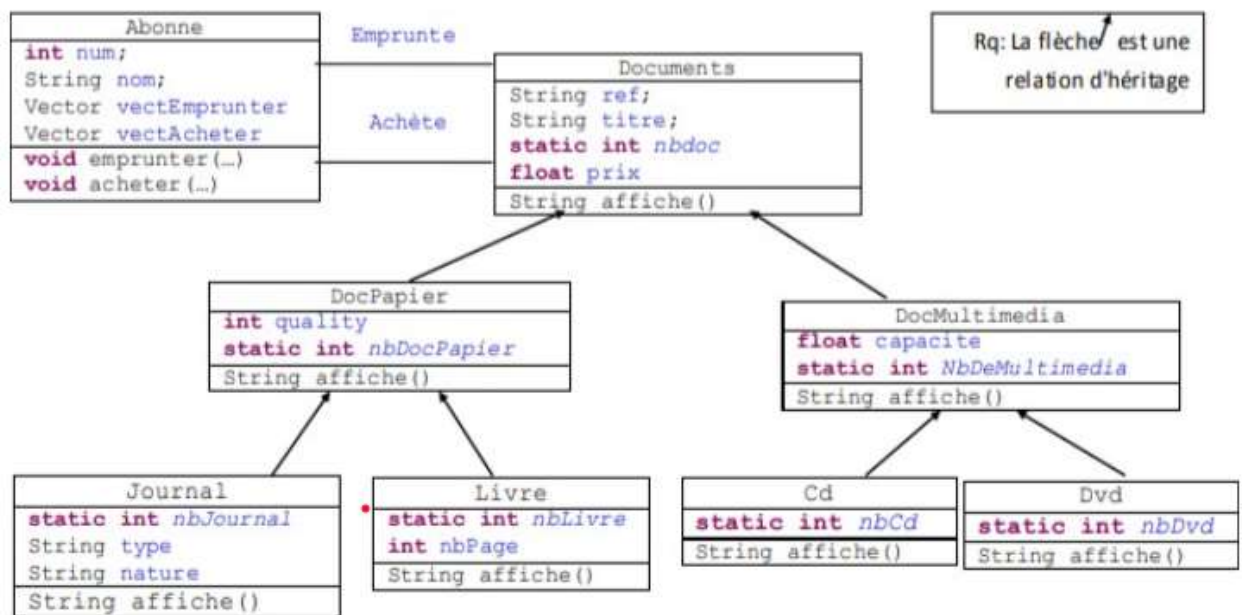
3. A partir de cette classe de base, créer les classes Chat et Chien qui héritent de la classe Animal et leur définir chacune un constructeur.

Redéfinir la méthodes jeMangeQuoi() respectivement pour les classes Chat et Chien (en suivant l'exemple de la méthode boire()).

- Créer un programme principal Test qui crée dans le main un objet « minou » de la classe Chat, le décrit (utiliser la méthode toString()) puis fait appel aux méthodes jeMangeQuoi(), boire().
- Ensuite créer un objet « fox » de la classe Chien et faire de même l'appel aux 3 méthodes.
- Créer un tableau (taille 5) d'animaux qui contient 2 chats et 3 chiens puis faire l'appel aux 3 méthodes.

Exercice 2 :

Soit le diagramme de classe suivant pour la gestion d'une bibliothèque. La bibliothèque contient actuellement deux sortes de document : un document papier (un livre ou un journal) ou un document multimédia (comme un CD ou un DVD). Un abonné peut emprunter ou acheter plusieurs types de documents. Proposer alors une bonne gestion de cette bibliothèque en utilisant l'héritage de la programmation orientée objet.



- Proposer plusieurs constructeurs (Utiliser `This()` et `Super()` dans les constructeurs)
- Utiliser `Super` dans l'appel de méthode
- Implémenter ce diagramme.